

Distributed Systems: examples and trends

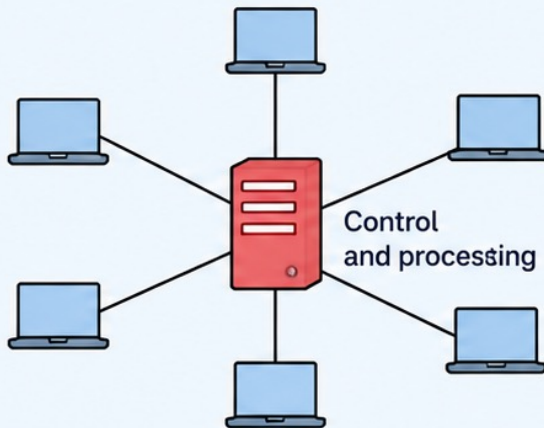
Distributed software systems

CdLM Informatica - Università di Bologna

Agenda

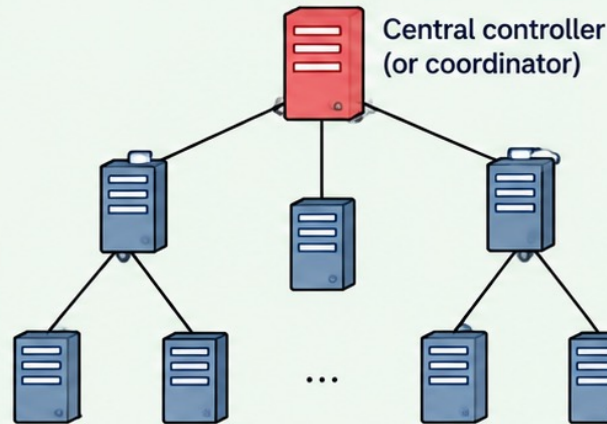
- Distributed systems: examples
- Challenges
- Case studies

Centralized system (single computer)



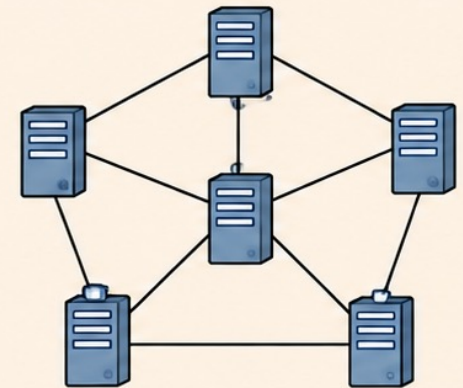
- Computation and data on a single computer
- Single point of control
- Single point of failure
- Simple to manage (but limited scalability)

Distributed system (multiple computers, possibly centralized control)



- Computation and data distributed across multiple computers
- Decisions may be centralized
- Nodes may have a master/coordination node
- More scalable and fault tolerant than a single computer

Decentralized system (distributed and no single point of control)



- Computation and data distributed across multiple computers
- No single point of control
- Nodes make local decisions based on partial information
- More resilient, but harder to manage

More centralized control

More decentralized control

A decentralized system is always a (subset of a) distributed system.

A distributed system may be centralized or decentralized.



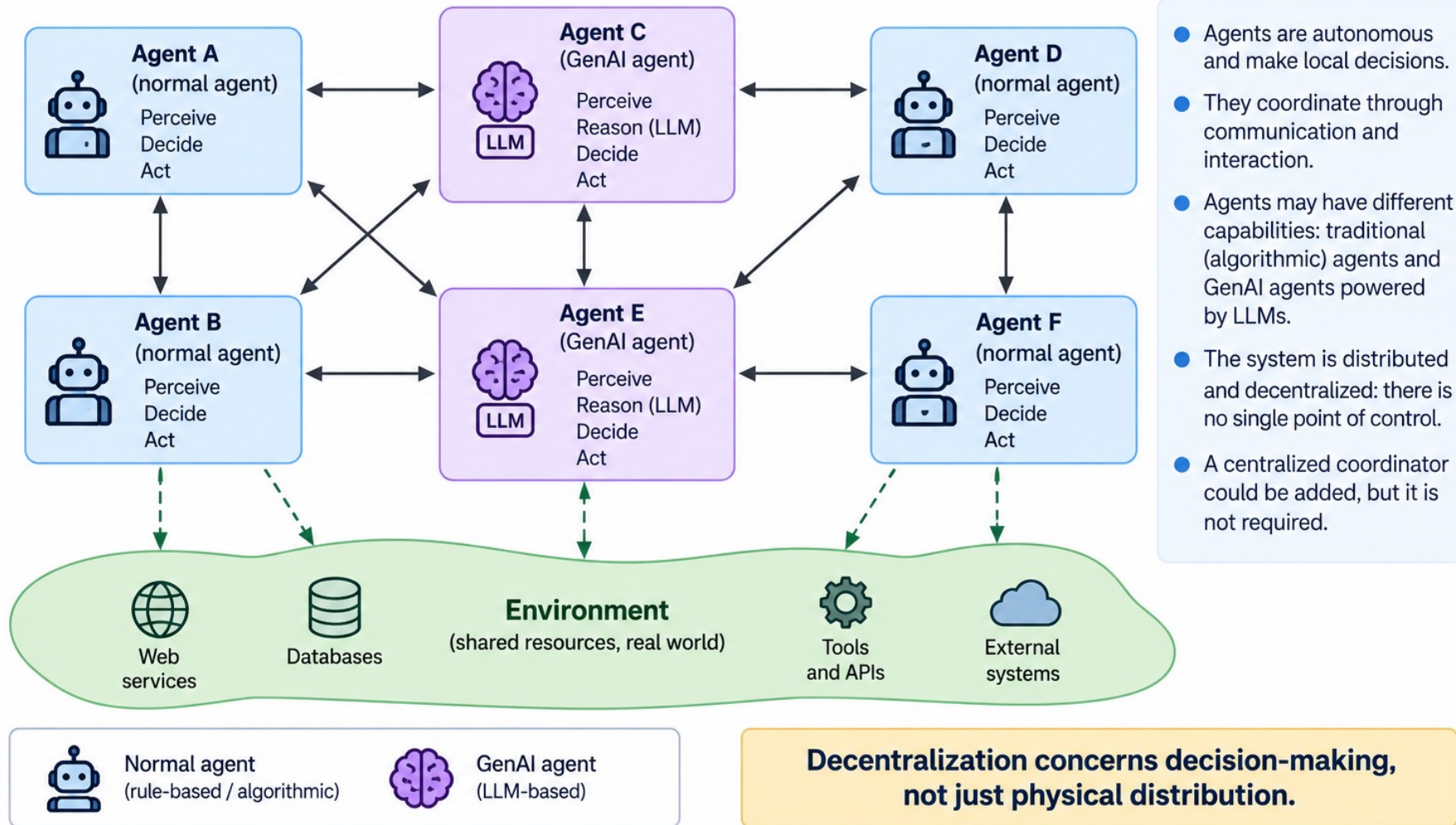
Control node
(or coordinator)



Worker/peer node

Multi-Agent Systems: a Decentralized Example

Autonomous agents (both conventional and GenAI-based) interact and coordinate to achieve a common goal, without a single point of control.



Centralized vs distributed: important points

Centralized

- Only one component with non-autonomous parts
- Component shared by users all the time
- All resources accessible
- Software runs in a single process (possibly with simulated concurrent threads)
- Single point of **control**
- Single point of **failure**

Decentralized

- Multiple autonomous components
- Components are not shared by all users
- Some resources may not be accessible
- Software runs in really concurrent processes on different processors
- Multiple points of control
- Multiple points of failure

Definition: Scalability of a system

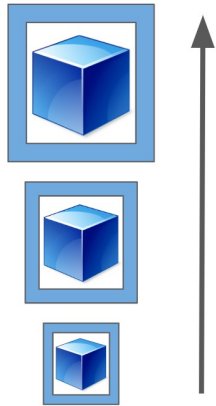
- **Definition: scalability**

- the ability of a system to handle increasing amounts of work or traffic without sacrificing performance
- A scalable system can easily accommodate more users or data without becoming overloaded

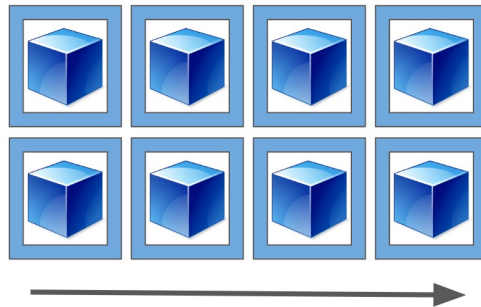
- **Related property: elasticity.**

It is the ability of a cloud to automatically expand or compress the infrastructural resources on a sudden up and down in the workload, so that it can be managed efficiently. The usage of elastic resources helps a company to minimize infrastructural costs, as elasticity is automatically scaling up or down resources to meet user demands.

Vertical Scaling
(increased instance size)



Horizontal Scaling
(increased instance count)



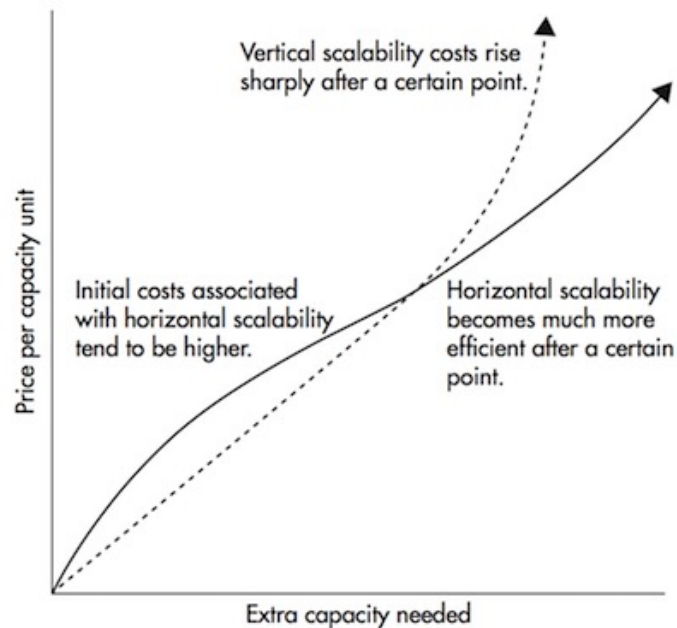
Why distribute a computing system?

The only way to handle more traffic with a single computer consists of upgrading the hardware: this is called **scaling vertically**.

Scaling vertically is good, but after a certain point you will see that even the best hardware is not sufficient for traffic, and impractical to host.

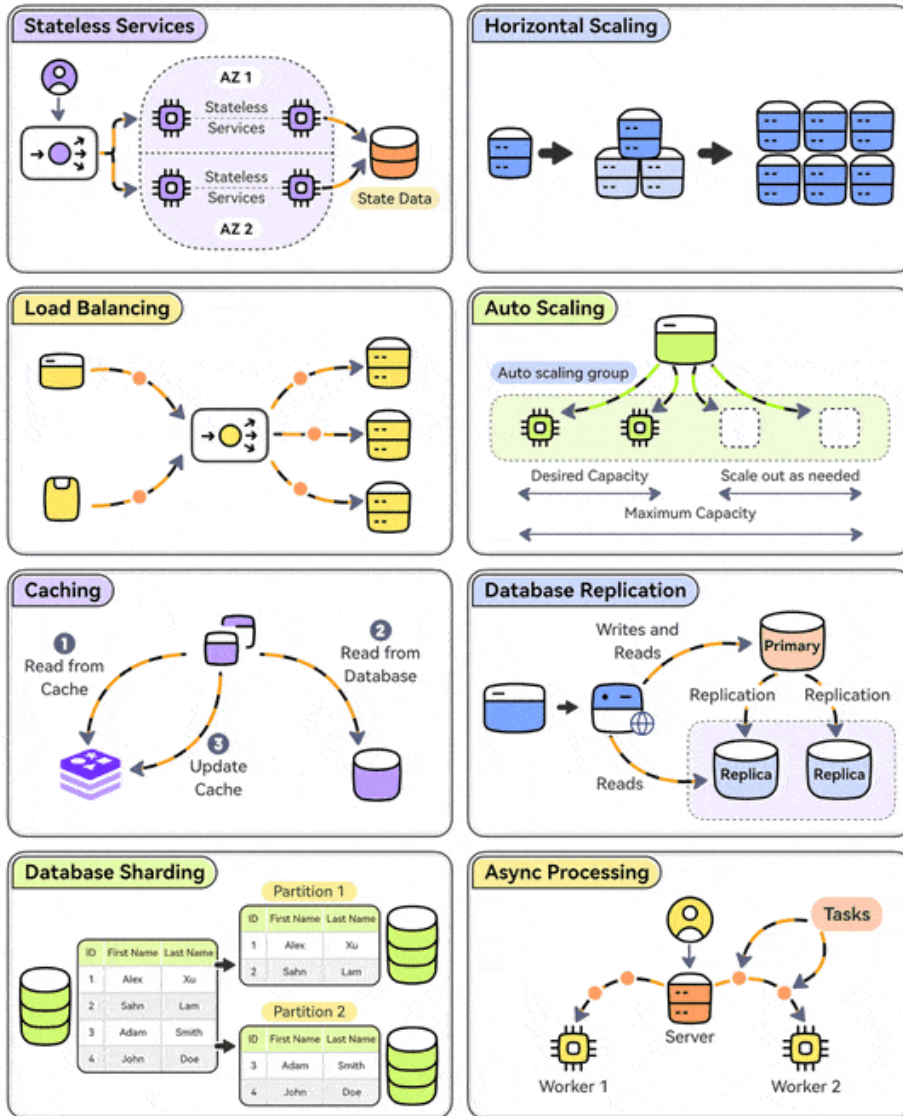
A distributed system can scale horizontally.

Scaling horizontally simply means adding more computers rather than upgrading the hardware of a single one.



8 Must Know Strategies to Scale Your System

ByteByteGo



1. **Stateless Design** stateless services because they don't rely on server-specific data and are easier to scale.
 2. **Horizontal Scaling** Add more servers so that the workload can be shared.
 3. **Load Balancing** Use a load balancer to distribute incoming requests evenly across multiple servers.
 4. **Auto Scaling** Implement auto-scaling policies to adjust resources based on real-time traffic.
 5. **Caching** Use caching to reduce the load on the database and handle repetitive requests at scale.
 6. **Database Replication** Replicate data across multiple nodes to scale the read operations while improving redundancy.
 7. **Database Sharding** Distribute data across multiple instances to scale the writes as well as reads.
 8. **Async Processing** Move time-consuming and resource-intensive tasks to background workers using async processing to scale out new requests.
- Which other strategies have you used?

Definition: Availability of a system

Availability: the percentage of time a system is up in a timeframe

Availability class	%	Downtime per year	Downtime per month	Downtime per day
Two nines	99%	3.65 days	7.31 hours	14.40 minutes
Three nines	99.9%	8.77 hours	48.83 minutes	1.44 minutes
Five nines	99.999%	5.26 minutes	26.30 seconds	864 milliseconds
Seven nines	99.99999%	3.16 seconds	262.98 milliseconds	8.64 milliseconds

Definition: Resiliency

Resiliency: the ability of a system to recover from failures or disruptions.

The key strategies to obtain resilient systems:

- Redundancy
- Fault tolerance
- Monitoring and testing
- Disaster recovery

Distributed systems in enterprises

- Large businesses and enterprises are decentralized, with multiple users, services, and huge big data lakes
- Distributed computing moves information and services closer to users who need them
- Personal and mobile client computers connected to network servers are less expensive and more available and resilient than mainframe computers
- However, the **total cost of ownership** is still expensive when there are many many servers
- Solution: **cloud computing** (centralization again!)

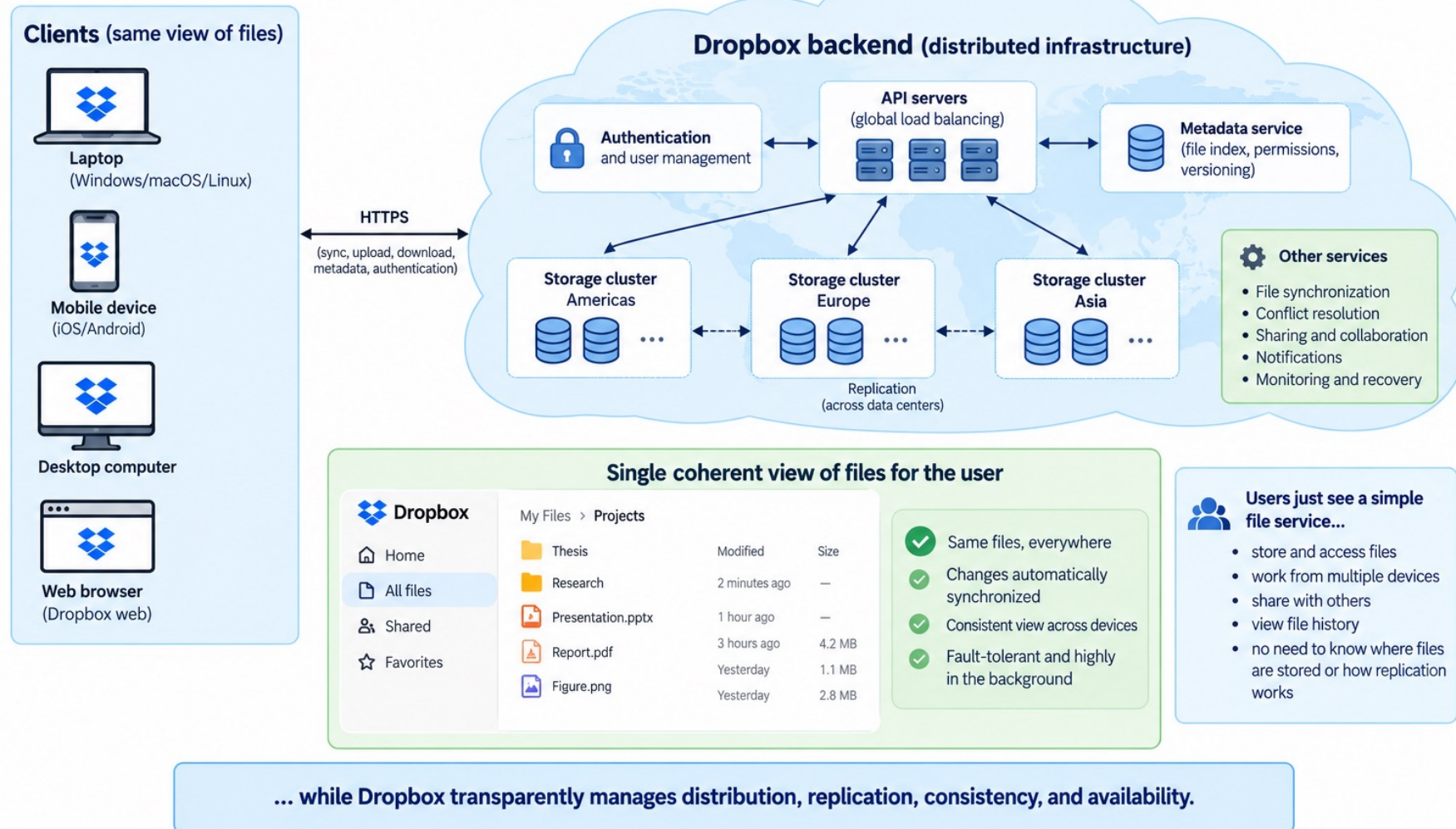
What is a distributed system

*A distributed system is
a collection of autonomous computing elements
that appears to its users
as a single coherent system*

(note that this is a definition emphasizing transparency!)

Dropbox: a distributed application

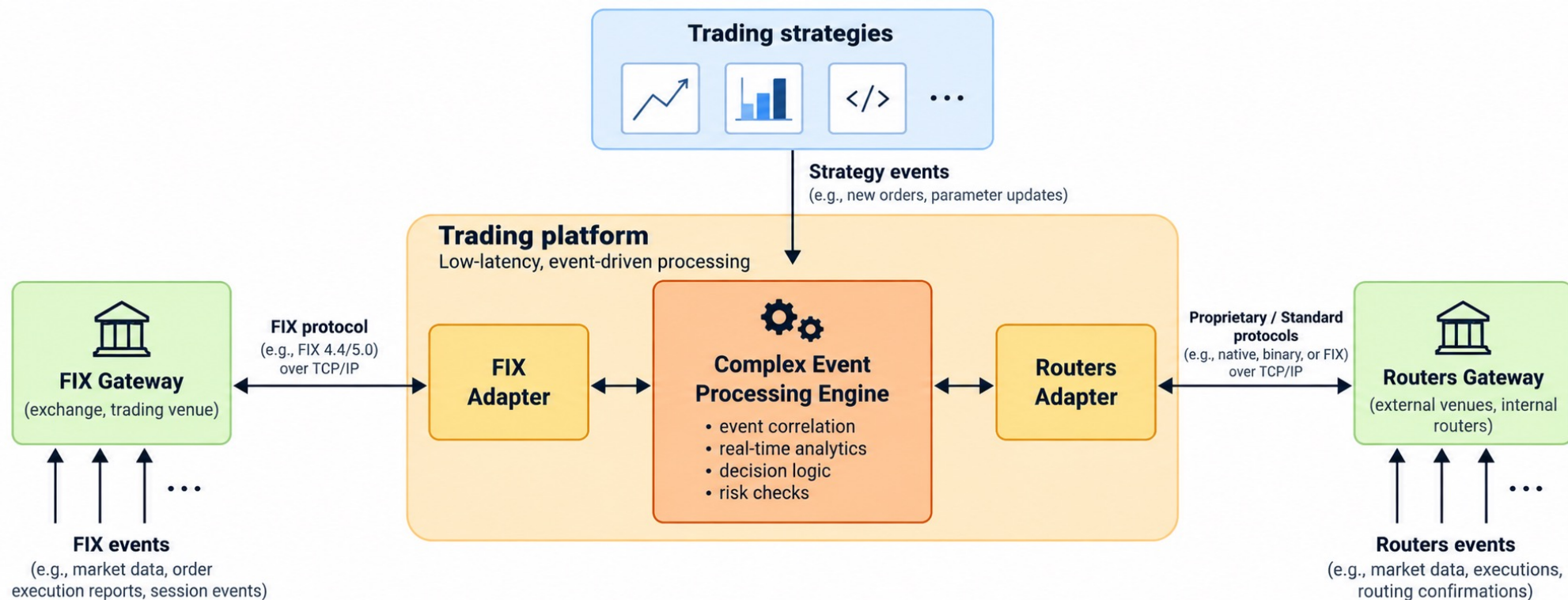
Users see a single, coherent view of their files, while data and services are distributed across multiple servers and data centers.



Single coherent system: features

- **Single system view**, behaving according to the expectations of users, that can be ubiquitous
- The collection of nodes **as a whole** operates the same, no matter where, when, and how user-system interactions take place
- Data can be replicated: **distribution transparency**
- **Middleware** to support programming and deploying distributed applications

An example financial trading system



Can all users observe the same market state at the same instant?

Important non functional property: **simultaneity** (very difficult to test)

See <https://queue.acm.org/detail.cfm?id=2745385>

Collection of autonomous computing elements

In a distributed system nodes act **independently** from each other

- They are programmed (by different people, with different technologies) by exchanging **messages**
- Each node can **fail** independently from others: **partial** failures are normal
- Each node has its own time (**the system has no global clock**)

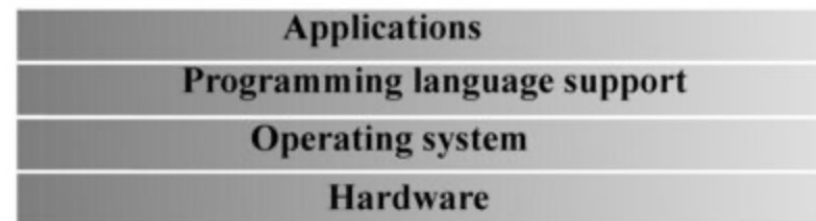
Distributed systems vs networks of computers

- A **computer network** is an interconnected collection of autonomous computers able to exchange data.
- A computer network usually requires users to **explicitly** login on one machine, **explicitly** submit jobs remotely, **explicitly** move files/data around the network.
- In a **distributed system** the existence of multiple autonomous computers is **transparent** to the user.
- The **middleware** automatically allocates jobs to processors, moves files among various computers without explicit user intervention

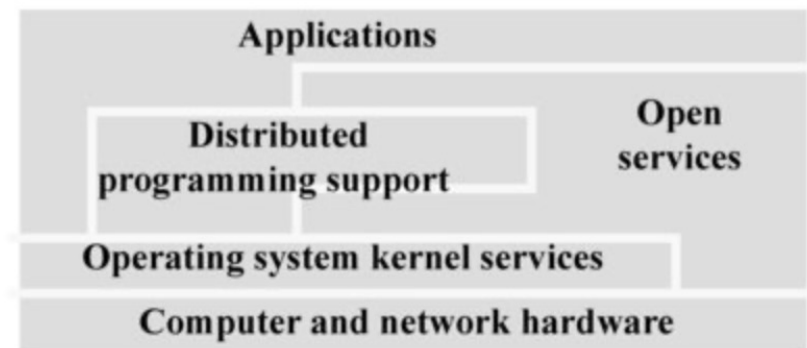
The software structure of distributed systems

- Network protocols
- IPC mechanisms
- Overlay networks
- Middleware
- API and services

Centralized
System



Distributed
System



Overlay network

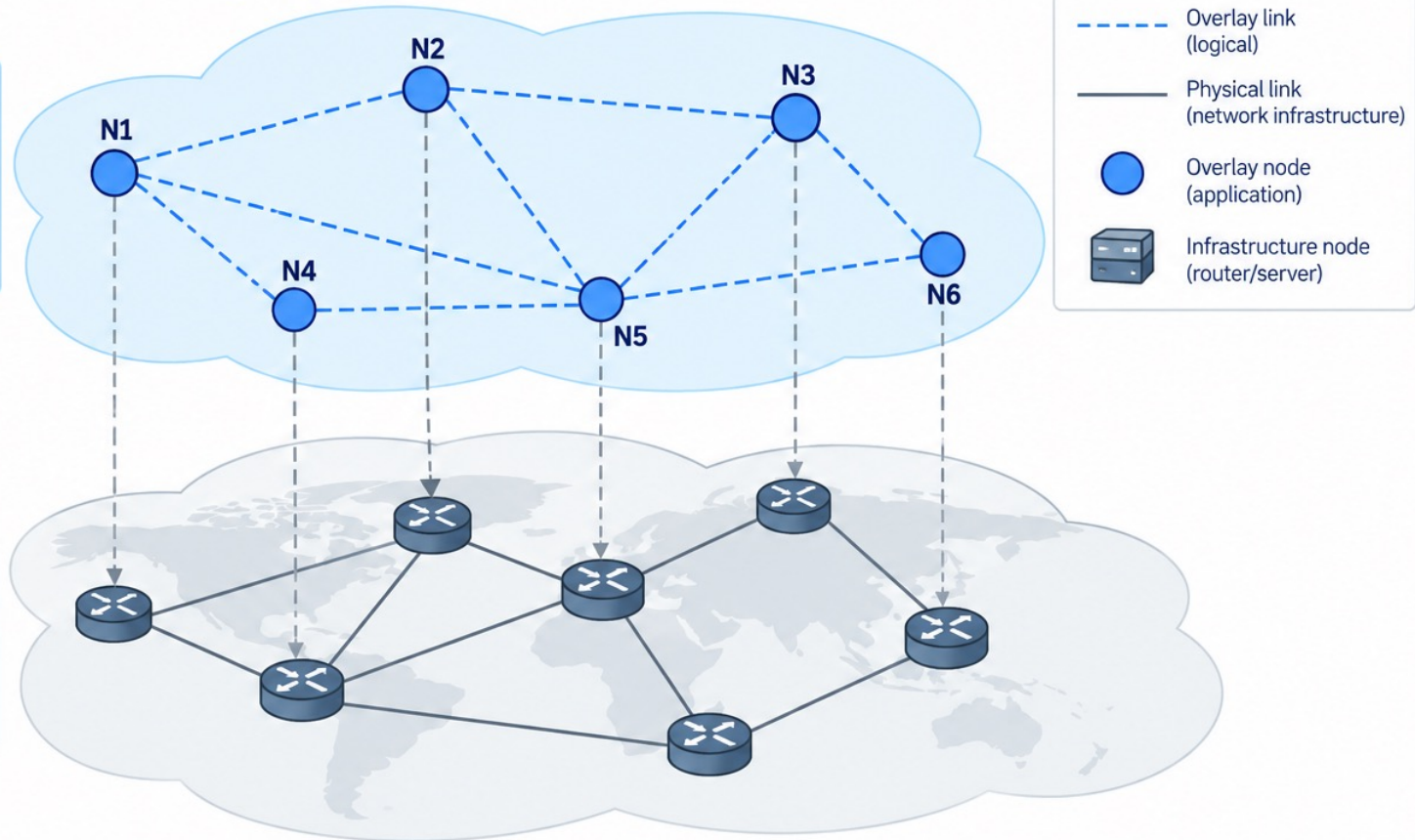
An overlay network is a virtual network built on top of an existing network infrastructure, connecting application-level nodes through logical links.

Overlay network (application layer)

Logical links between overlay nodes (e.g., for content distribution, P2P, or trust management)

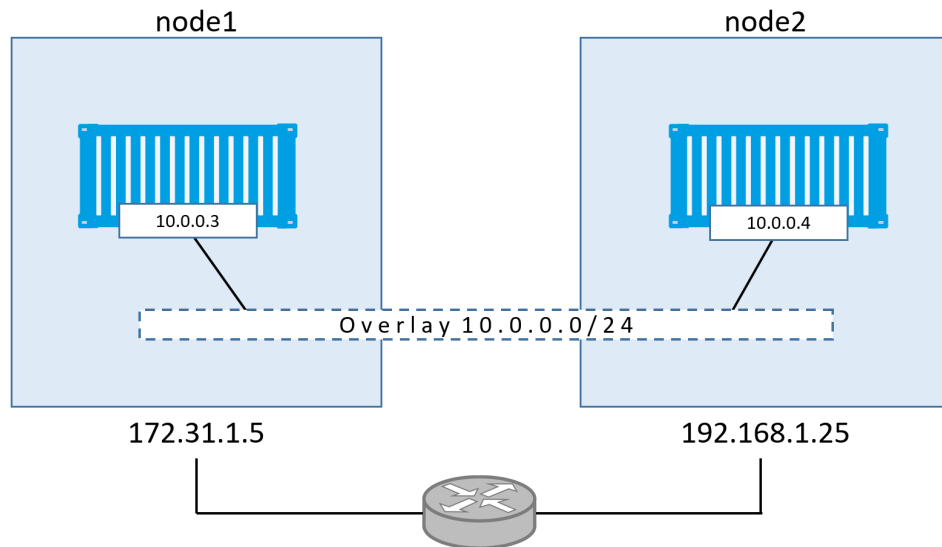
Underlying network (physical infrastructure)

Routers and links provided by the Internet or a private network



Overlay networks create application-level connectivity on top of an existing network, abstracting from the physical topology and enabling flexible, decentralized services.

Overlay networks



- Each node in an overlay network communicates only with other nodes in the same network, its *neighbors*.
- The set of neighbors may be dynamic, or may even be known only implicitly (i.e., it requires a lookup).
- Well-known example of overlay network: **peer-to-peer systems**.
- Overlay network types
 - **Structured**: each node has a well-defined set of neighbors with whom it can communicate (tree, ring).
 - **Unstructured**: each node has references to randomly selected subset of the nodes from the system

Scalability

Scalability is the capability of a system to handle a growing amount of work

- For example, a system is considered scalable if it is capable of increasing its total output under an increased load when resources (hardware) are added.

A system whose performance improves after adding hardware, proportionally to the capacity added, is said to be a **scalable system**.

- An algorithm, design, networking protocol, program, or other system is said **to scale** if it is suitably efficient and practical when applied to large situations (e.g. a large input data set, a large number of outputs or users, or a large number of participating nodes in the case of a distributed system).
- If a system fails when a quantity increases, it does not scale.

Scaling

- Horizontal scaling: you increase the number of machines
- Vertical scaling: you increase the performance of your system by increasing the performance of one machine
- Scaling is limited by Amdahl's law

The Amdahl law

The Amdahl law is a formula which gives the theoretical speedup in latency of the execution of a task at **fixed workload** that can be expected of a system whose resources are improved. It is often used in distributed computing to predict the theoretical speedup when using multiple processors.

$$S_{\text{latency}}(s) = \frac{1}{(1 - p) + \frac{p}{s}}$$

where:

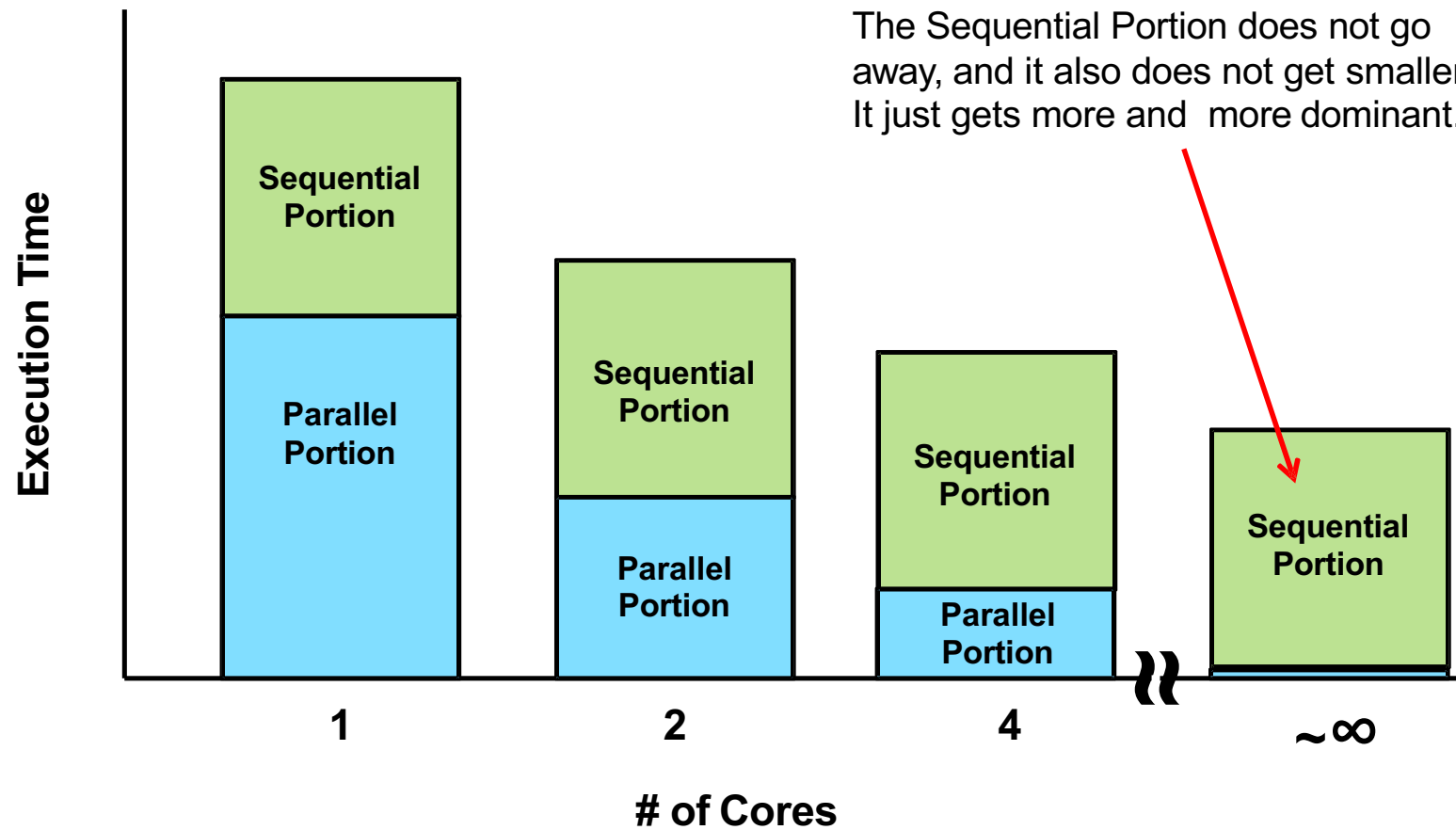
- S_{latency} is the theoretical improvement of the execution time (*latency*) of the complete task
- s is the improvement in execution (*speedup*) of the part of the task that benefits from the improved system resources.
- p is the proportion of the execution time that the part benefiting from improved resources actually occupies.

Example (source: Wikipedia, article on Amdahl law)

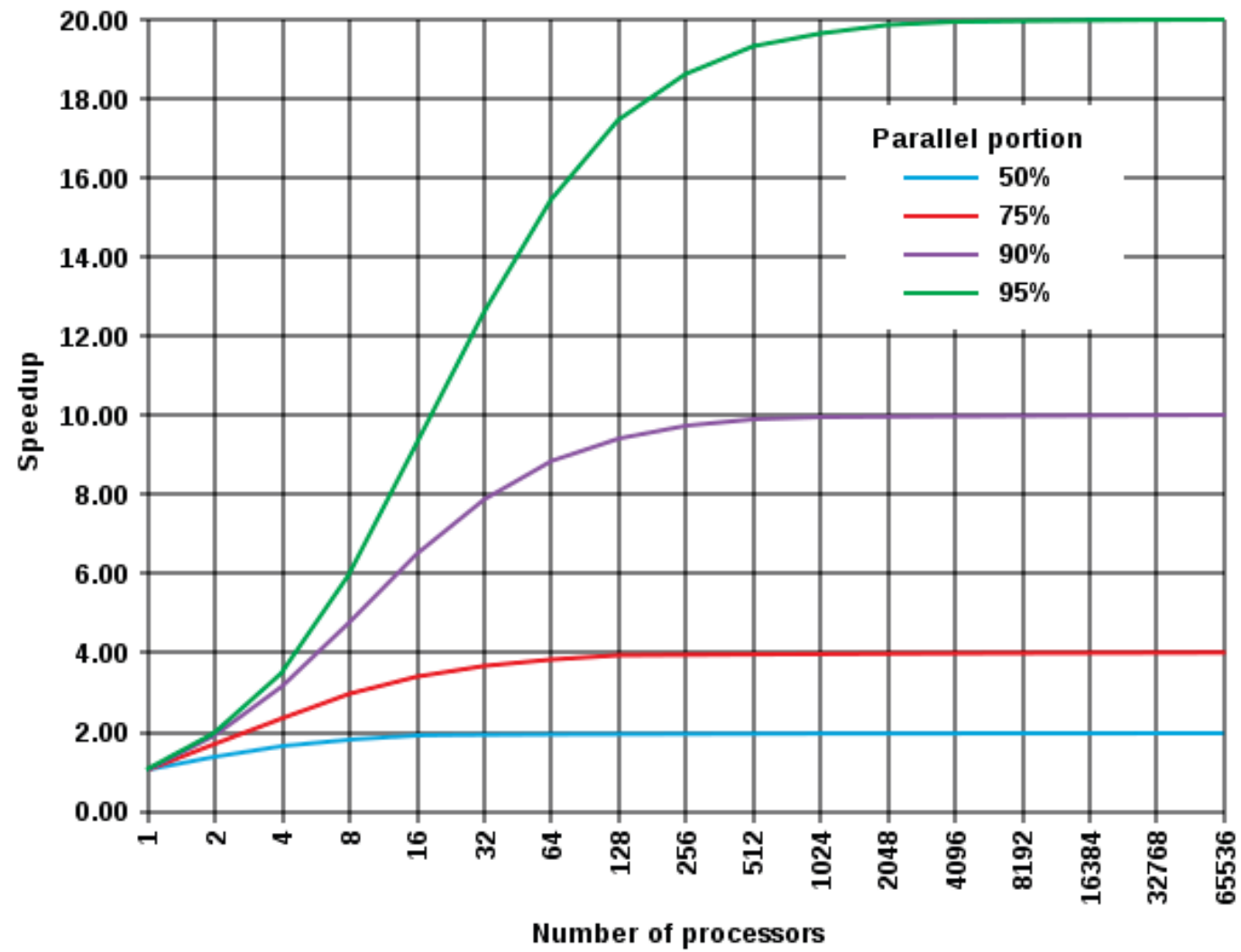
If a program needs 20 hours using a single processor core, and a particular part of the program which takes one hour to execute cannot be parallelized, while the remaining 19 hours ($p = 0.95$) of execution time can be parallelized, then regardless of how many processors are devoted to a parallelized execution of this program, the minimum execution time cannot be less than that critical one hour.

Hence, the theoretical speedup is limited to at most 20 times ($1/(1 - p) = 20$). For this reason, computing with many processors is useful only for highly parallelizable programs

A Visual Explanation of Amdahl's Law



Amdahl's Law



Consequences of the Amdahl's law

- An implication of Amdahl's law is that to speedup real applications which have both serial and parallel portions, heterogeneous computing techniques are required.
- For example, a CPU-GPU heterogeneous processor may provide higher performance and energy efficiency than a CPU-only or GPU-only processor
- Amdahl's law applies only to the cases where the problem size is fixed. In practice, as more computing resources become available, they tend to get used on larger problems (larger datasets), and the time spent in the parallelizable part often grows much faster than the inherently serial work. In this case, Gustafson's law gives a less pessimistic and more realistic assessment of the parallel performance.

Challenges of distributed systems

- Heterogeneity of computing resources
- Openness
- Security
- Scalability
- Failure handling
- Quality of Service
- Transparency
- Absence of absolute time

Challenge: Heterogeneity of resources

- Networks (both physical media and protocols)
- Computer hardware;
- Operating systems;
- Programming languages;
- Implementations by different developers;
- Use cases: eg mobility vs home computing
- Systems of systems

Using middleware: main services

Communication

RPC, REST APIs, gRPC, messaging

Transactions and consistency

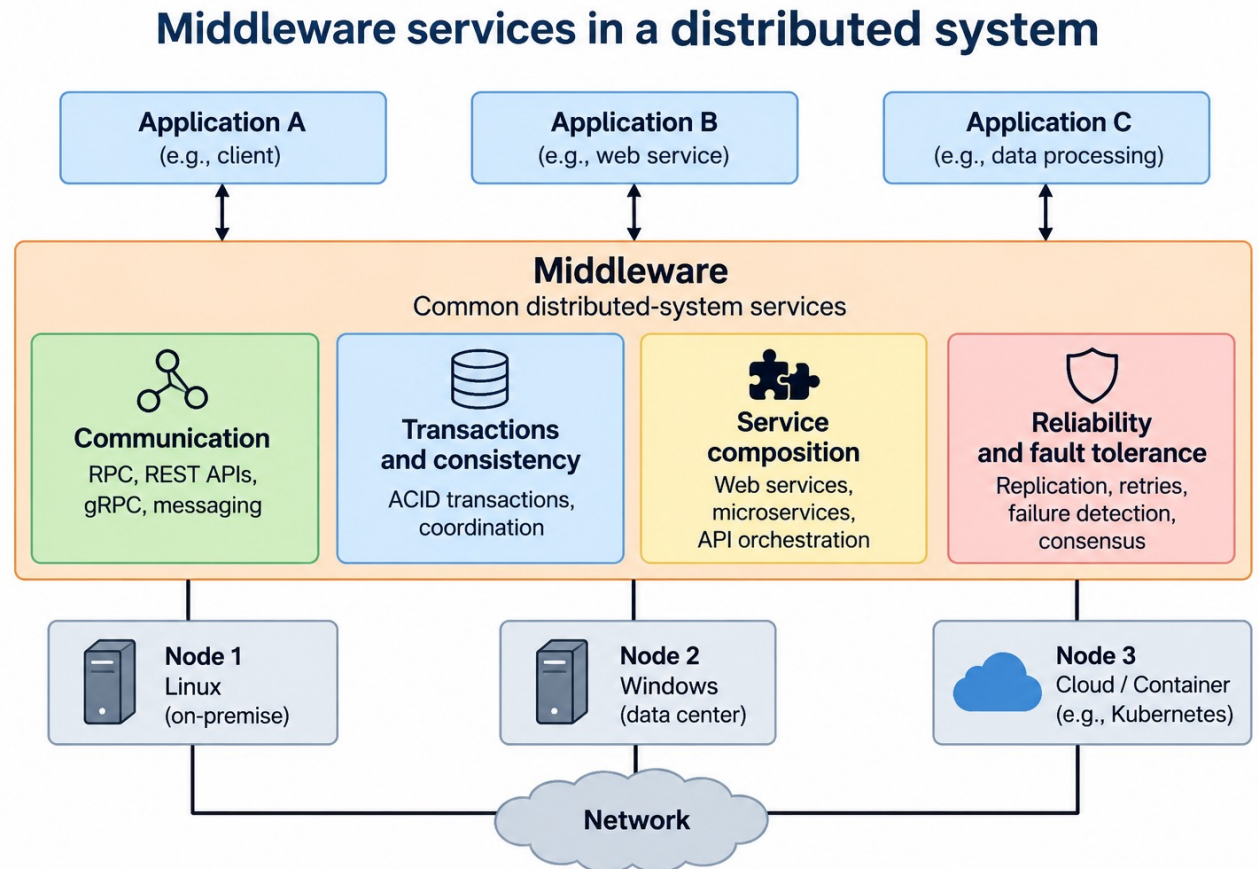
ACID transactions, distributed transactions

Service composition and coordination

Web services, microservices, API orchestration

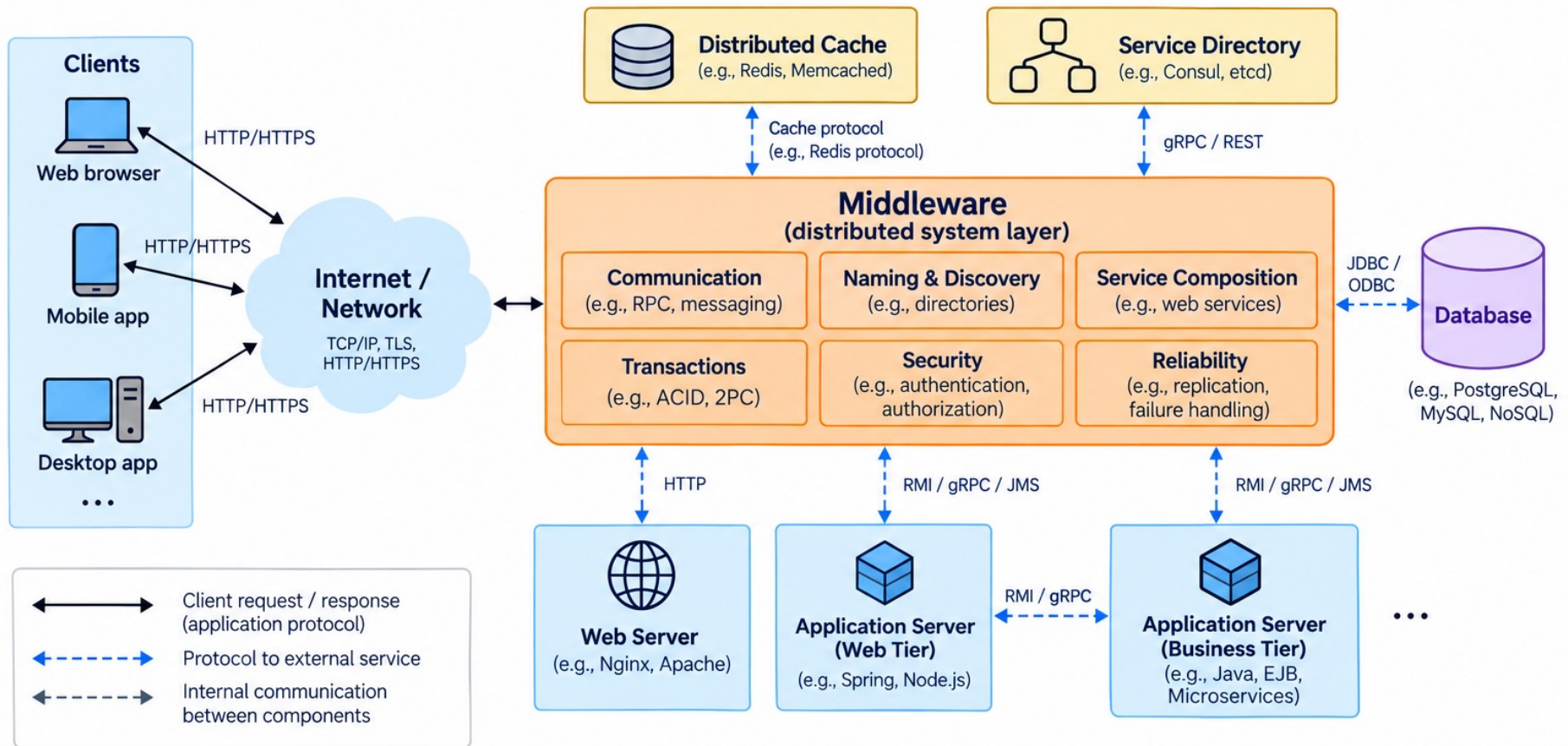
Reliability and fault tolerance

Replication, retries, failure detection, consensus



Where is the middleware?

Middleware is the software layer that provides common services to applications and hides the complexity of communication, location, security, transactions, etc.



The middleware is located between applications and the underlying network infrastructure, and provides common services through standard interfaces and protocols.

The Transparency dogma

Middleware should hide both topology (i.e. remoteness) and concurrency by **hiding distribution** behind local programming language constructs

Critique:

Full transparency is impossible, and the price is too high

Challenge: Openness

- Open systems have their key API public (API = Application Programming Interface)
- Open systems are based on the provision of a uniform communication mechanism and published interfaces for access to shared resources.
- Open systems can be constructed from heterogeneous hardware and software, possibly from different vendors.
- The conformance of each component to the published standard (we call this **interoperability**) must be carefully tested and verified if the system is to work correctly.

Challenge: Scalability

Scalability is the property of a system to handle a growing amount of work by adding resources to the system

- Controlling the cost of physical resources
- Controlling the performance loss
- Preventing software resources running out
- Avoiding performance bottlenecks

Challenge: Failure handling

Computer systems sometimes fail.

When faults occur in hardware or software, programs may produce incorrect results or may stop before they have completed the intended computation

- Detecting failures (eg. Checksum)
- Masking failures (eg. retransmit msgs, data replication)
- Tolerating failures
- Recovery from failures
- Redundant components

Network transparency

- The two most important transparencies are *access* and *location* transparency: they strongly affect the usage of distributed resources
- They are referred to together as *network transparency*
- *Network transparency* is the situation in which an operating system allows a user to access a resource (such as some data or an application program) without the user needing to know or being aware if the resource is located on the local machine or on a remote machine (i.e., a computer elsewhere on the network).

Absence of absolute time

- A distributed system has no global management of time
- This is both a theoretical problem and a practical challenge:
 - distributed algorithms cannot exploit any notion of centralized clock
 - distributed applications must manage time-related operations in very specific ways

Conclusions: trends

Distributed systems technologies are continuously evolving.

Current trends include:

- Cloud-native computing: containers, microservices, orchestration, serverless, elasticity
- Edge and ubiquitous computing: computation closer to users and mobile devices
- IoT and cyber-physical systems: large numbers of connected, autonomous devices
- AI-native distributed systems: distributed training, inference, agentic systems, AI services
- Data-intensive computing: large-scale storage, stream processing, distributed analytics
- Resilience and geo-distribution: multi-region deployment, replication, fault tolerance
- Heterogeneous infrastructures: cloud, edge, accelerators, GPUs, specialized hardware

Exercise: scalability

A distributed service is experiencing a rapid increase in users.

- What does it mean for the system to be **scalable**?
- Give one example of **vertical scaling** and one of **horizontal scaling**.
- Which bottlenecks may prevent performance from improving when resources are added?

Exercise

- How might the clocks in two computers that are linked by a local network be synchronized without reference to an external time source?
- What factors limit the accuracy of the procedure you have described?
- How could the clocks in a large number of computers connected by the Internet be synchronized?
- Discuss the accuracy of that procedure.

Cristian's protocol

- The round trip time t to send a message and a reply between computer A and computer B is measured by repeated tests; then computer A sends its clock setting T to computer B. B sets its clock to $T + t / 2$. The setting can be refined by repetition.
- The procedure is subject to inaccuracy because of contention for the use of the local network from other computers and delays in the processing the messages in the operating systems of A and B. For a local network, the accuracy is probably within 1 ms.
- For a large number of computers, one computer should be nominated to act as the time server and it should carry out Cristian's protocol with all of them. The protocol can be initiated by each in turn.
- Additional inaccuracies arise in the Internet because messages are delayed as they pass through switches in wider area networks.
- For a wide area network the accuracy is probably within 5-10 ms. These answers do not take into account the need for fault-tolerance